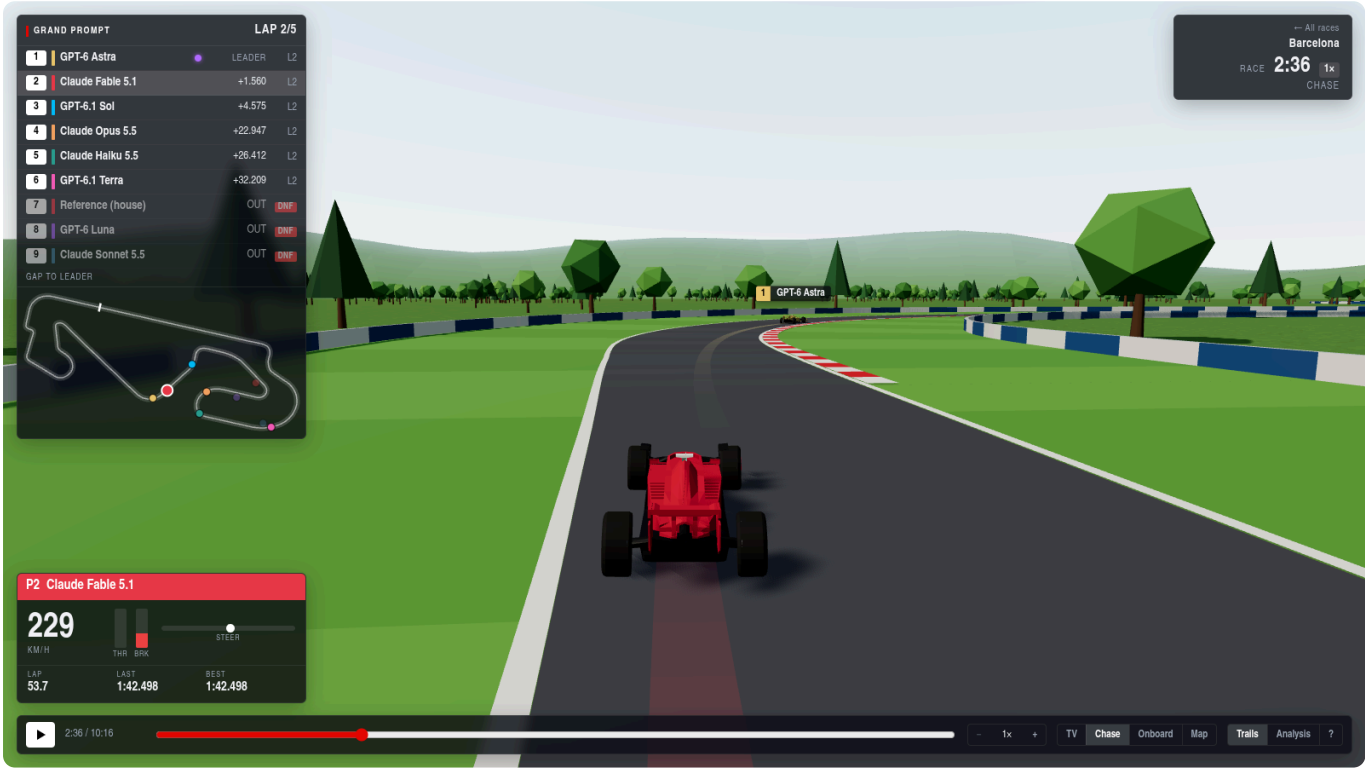
GRAND PROMPT

Design brief & product specification

A motorsport benchmark for LLMs: each model writes a driving program once, every program races the same car in a deterministic simulator, and the race is replayed in 3D.



Document	Cahier des charges for Claude Design — current state, constraints, requirements, open questions
Product	“Grand Prompt” (working title; repo `grand-prompt`, plan title “F1 LLM Bench”)
Date	9 November 2026 · sim 0.1.0 · driver API 0.1 · replay format 1
Owner	David Courtey
Audience	Claude Design (system design proposal), then the implementing coding agent

0 How to read this document

This brief describes what already exists, what is firm, and what is open. It is written so that Claude Design can return a **system design**: a visual design system plus screen and flow designs, and, where it sees fit, proposals for refactors and new features. Everything proposed will be reviewed by the owner before a coding agent implements it.

> **Freedom:** everything not listed as *firm* in §8 may be challenged: information architecture, screens, navigation, visual identity, component structure, even the split between apps. Proposals that break a firm constraint should say so explicitly and explain why.

1. Product in one page
2. Current status
3. System architecture
4. Benchmark pipeline & data produced
5. Viewer: current features (screenshots)
6. Current visual language
7. Known issues and gaps
8. Firm constraints
9. Requirements (cahier des charges)
10. Open areas: proposals welcome
11. Expected deliverables from Claude Design
12. Appendix: shortcuts, URL params, data schemas, glossary

1 Product in one page

What it is. A benchmark that turns “which LLM is the better engineer?” into a race people want to watch and share. Inspired by a sailing race where LLMs steered a yacht live, with one deliberate difference: **code as policy**. The model does not drive in the loop. It writes a program once (iterating during a limited practice), and that program drives at 50 Hz with no API calls during the race. Races are cheap, fast and bit-for-bit reproducible, and the benchmark measures physics reasoning, algorithms (racing line, speed profile, path tracking) and code quality.

Two halves

- **The engine:** a headless deterministic racing simulator (open-wheel car, bicycle model, Pacejka tyres, aero), race control (laps, sectors, track limits, penalties, DNFs), and a 3D replay viewer.
- **The Driver Development Kit (DDK):** everything an LLM receives (spec, TypeScript types, baseline driver, prompts, practice client) and the harness that builds, sandboxes, races and scores the programs.

Audiences

WHO	WHAT THEY NEED
People seeing a shared clip or link (often on a phone)	Instantly readable story: who is ahead, by how much, who crashed and why, what each model cost. Broadcast feel, not a dev tool.
AI-curious / technical readers	Leaderboard across events, per-model detail (tokens, cost, wall time, practice runs, effort), speed-trace comparison, reproducibility proof.
The owner (operator)	Browse every run (practice, qualifying, race), debug drivers, compare builds, prepare content to publish.

Goals (firm)

- A car model where braking points, racing line and corner speed visibly matter.
- Any LLM can produce a working driver from the DDK alone.
- Several drivers on one circuit, compared on lap time, shown together in a pleasant 3D replay.
- Every result reproducible from its manifest.

Non-goals for now: high-fidelity 3D physics, online multiplayer, user accounts, public leaderboard *website* (a design for one is welcome as a proposal), any official F1 branding.

---

2 Current status

AREA	STATE	NOTES
Simulator, race control, replays	DONE	500 Hz physics, 50 Hz driver, golden-hash determinism tests.
QuickJS sandbox, budgets, error policy	DONE	Deterministic op budgets; 3 strikes = `DNF: ECU FAILURE`.
DDK, practice service, workspaces	DONE	Practice through a local HTTP service with per-token run budgets.
Agentic builds via vendor CLIs	DONE	`claude -p` / `codex exec` in a bubblewrap sandbox, 30 min, 10 practice runs.
Circuits	5	Prototype Ring (handmade), Monaco, Silverstone, Portimão (practice), Barcelona (unseen event circuit).
Events run	2	`proto-gp-1` (4 Claude models, trial), `gp-1` (8 models on Barcelona, before effort was fixed).
3D replay viewer	MVP+	Race list, 4 cameras, HUD, analysis, classification. Imported low-poly car in progress.
House reference driver	WEAK	Far off the pace, crashes on real circuits; removed from races by default.
Video export (Remotion), mobile viewer	NOT STARTED	Phase 4.
Wheel-to-wheel racing	NOT STARTED	Phase 5: shared world, collisions, slipstream, tyre wear.

Latest result: gp-1, Barcelona, 5 laps

POS	MODEL	QUALI	RACE	STATUS	COST	TOKENS	WALL
1	GPT-6 Astra	1:36.962	8:09.849	Finished	–	2,175k	14m42
2	Claude Fable 5.1	1:38.107	8:14.906	Finished	$3.83	1,277k	11m27
3	GPT-6.1 Sol	1:39.570	8:23.195	Finished	–	1,239k	5m47
4	Claude Opus 5.5	1:54.021	9:35.237	Finished	$0.59	649k	2m33
5	Claude Haiku 5.5	1:57.152	9:50.838	Finished	$0.02	522k	1m09
6	GPT-6.1 Terra	2:02.080	10:16.230	Finished	–	568k	1m33
DNF	GPT-6 Luna	–	–	Crashed	–	805k	2m30
DNF	Claude Sonnet 5.5	–	–	Crashed	$0.16	417k	0m57

Cost is reported by the Claude CLI only; Codex reports tokens but no cost. This table lives today only in CLI Markdown output (`bench leaderboard`), not in the viewer.

3 System architecture

```
 tracks/*.json ─┐                                    ┌─▶ replay.json.gz   (header + track + 50 Hz frames
+ events)
 cars/proto.json┼─▶  packages/sim  (pure TS)  ──────┼─▶ results.json     (classification, laps,
penalties)
                │    fixed timestep, race control   └─▶ telemetry.csv / summary.json (practice)
                │          ▲ step(telemetry) → controls (50 Hz)
 drivers/…/driver.js ─▶ packages/sandbox (QuickJS-WASM, op budgets, no host APIs)

 packages/cli  "bench": validate · practice · qualify · race · event run/reproduce · leaderboard ·
serve · workspace
 scripts/build-drivers.sh + models.json ─▶ vendor CLI in bubblewrap ─▶
drivers/<event>/<model>/{driver.js, manifest.json}

 apps/viewer  (Vite + React + react-three-fiber)  plays replays only, no physics
              serves results/ at /results/ and /results/index.json (built per request)
```

UNIT	ROLE
`packages/sim`	Browser-safe TS. Track spline + Frenet projection + surfaces, vehicle model, race control (`CarSim`), sessions (ghost mode), replay writer and the reference frame interpolation.
`packages/sandbox`	Runs untrusted driver code in QuickJS with deterministic compute budgets (init 20k units, step 20 units), seeded `Math.random`, no `Date`.
`packages/cli`	The `bench` command, the practice HTTP service, workspace creation, events and leaderboard (text/Markdown output, readable by humans and LLMs).
`packages/reference-driver`	Internal house driver; never shipped in the DDK.
`apps/viewer`	3D replay viewer. Pure logic in `src/lib` (unit tested: camera, timing, laps, minimap, messages, track mesh), scene in `src/scene`, HUD in `src/components`, dev-server middleware in `server/`.
`ddk/`	What an LLM sees: `DRIVER_SPEC.md`, `driver.d.ts`, baseline driver, public car params, launch/agentic/one-shot prompts, zero-dependency practice client.

Ghost mode (phases 1–4): every car runs alone in its own world from a standing start at s = 0; the replay overlays them. There is no contact between cars. This is honest (“separate runs, same conditions, shown together”) but creates a visual issue at the start (§7).

4 Benchmark pipeline & data produced

1 · Build
Each model (from `models.json`) runs its vendor CLI, sandboxed, 30 min, medium effort.

2 · Practice
10 runs shared across Monaco, Silverstone, Portimão via the practice service.

3 · Submit
`driver.js` + `manifest.json` (tokens, cost, wall time, runs, outcome).

4 · Qualifying
Out lap + 1 flying lap per driver on the unseen circuit (Barcelona).

5 · Race
N laps, ghost mode, seeded. Replay + results + state hash.

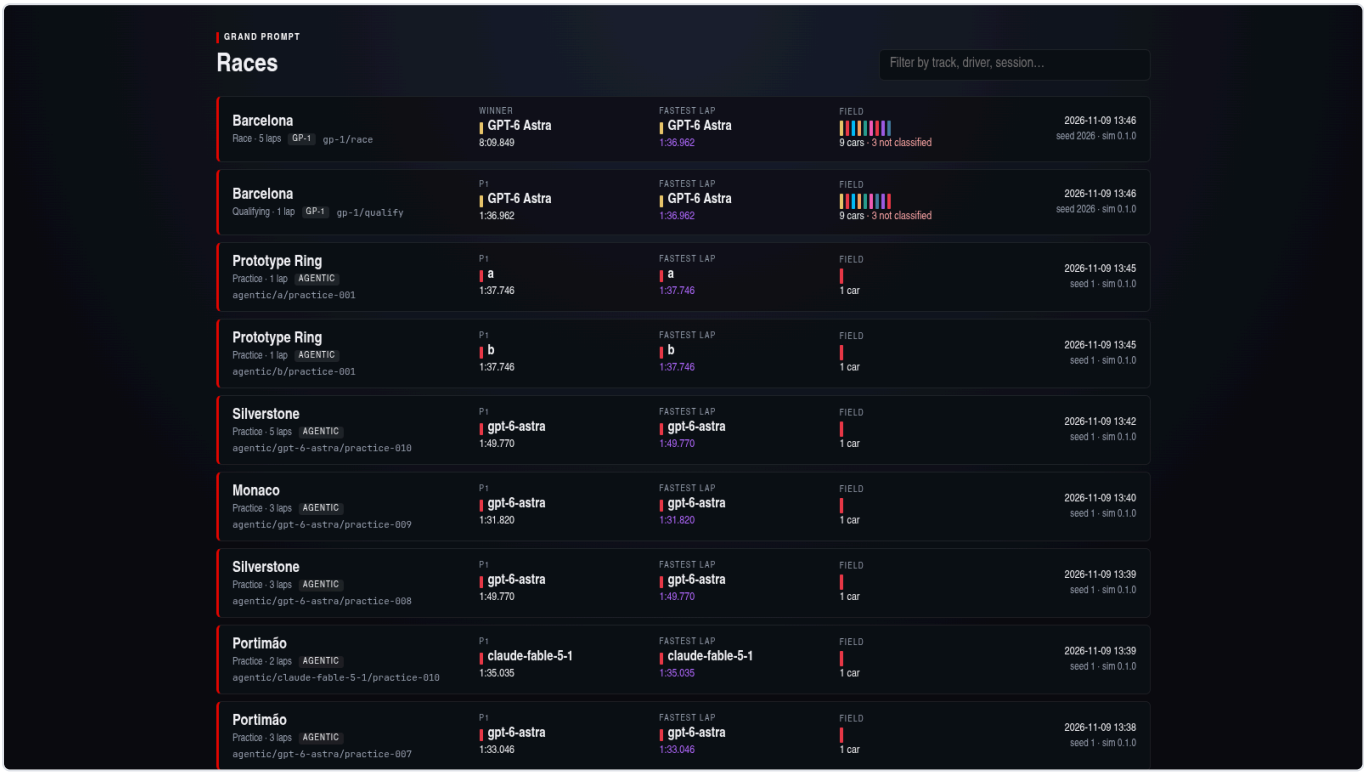
6 · Leaderboard
Points 10-8-6-5-4-3-2-1 across events; `event reproduce` re-checks hashes.

Data available to any UI (today, mostly unused by the viewer)

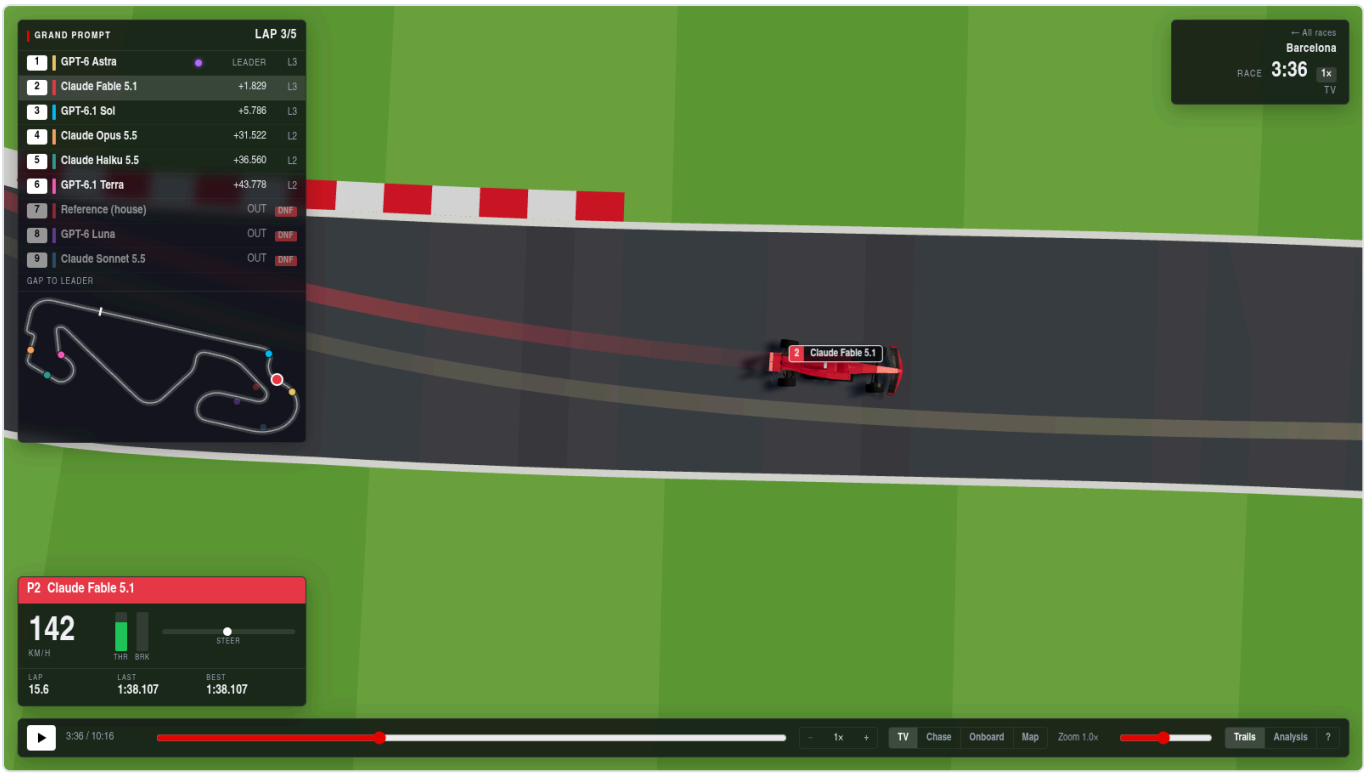
ARTEFACT	CONTENT	SHOWN IN VIEWER?
Replay header	sim / API / DDK versions, track & car hashes, seed, session type and laps, state hash, car dimensions, optional reference lap	PARTLY
Replay frames (50 Hz per car)	x, y, yaw, speed, steer, throttle, brake, surface, lap, s	YES
Replay events	lap/sector complete, lap deleted (track limits), fastest lap, warning, penalty, crash (speed), barrier impact, driver code fault, status change (finished / DNF)	YES (message feed)
Classification	position, total time, gap, best lap, penalties, status	YES
Track data	centreline, widths, kerbs, runoff type/width, sectors, curvature	YES (3D + minimap)
Manifest (per model)	model id, provider/CLI, effort, prompt & DDK version, input/output tokens, cost, wall time, turns, practice runs used, outcome, driver hash/source	NO
Build logs & practice history	each practice run (track, laps, times, replay), CLI transcript/logs, final driver code	NO (practice replays listed flat)
Event results & leaderboard	quali + race per model, points, wins, DNFs, total cost	NO (CLI only)
Telemetry CSV (practice)	s, speed, throttle, brake, steer, n, surface, ax, ay	NO

5 Viewer: current features

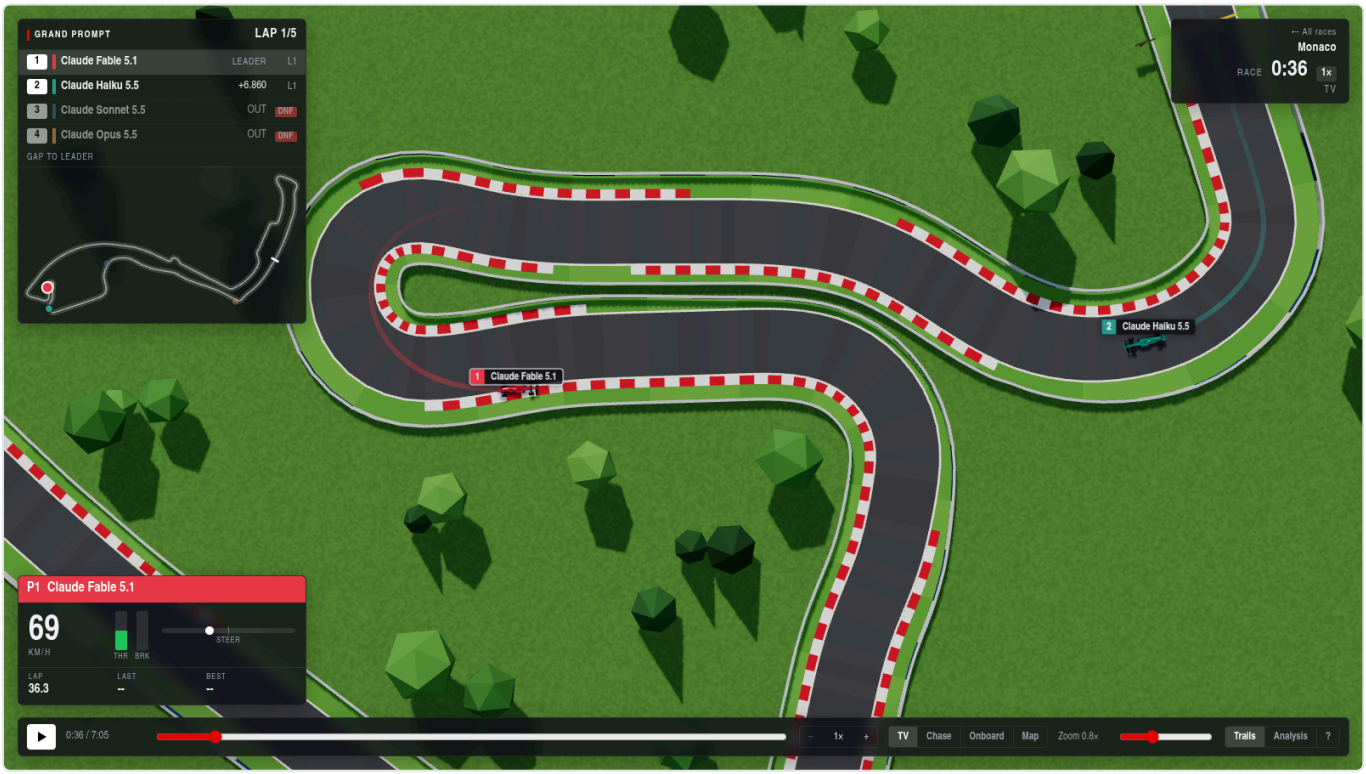
The viewer opens on a list of every replay under `results/`; a row opens a replay at `?replay=<url>`. Drag-and-drop of a `replay.json(.gz)` also works. Desktop-first, dark broadcast HUD over a low-poly 3D world.



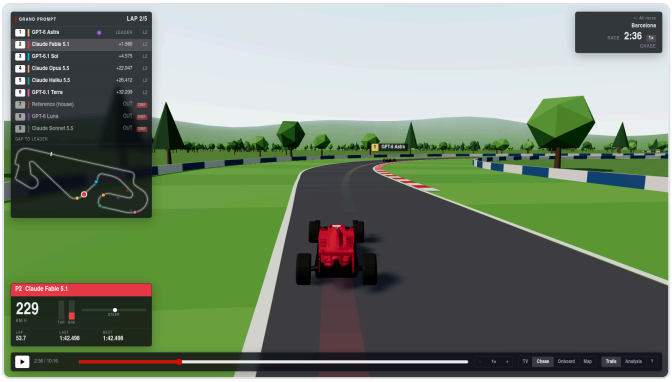
5.1 Race list (home). Newest first, filterable. Each row: track, session type and laps, group tag (event or AGENTIC practice), winner/P1 and time, fastest lap (purple), field as colour chips with “not classified” count, date, seed, sim version. Practice runs from agentic builds dominate the list: there is no grouping per event, no leaderboard and no model page.



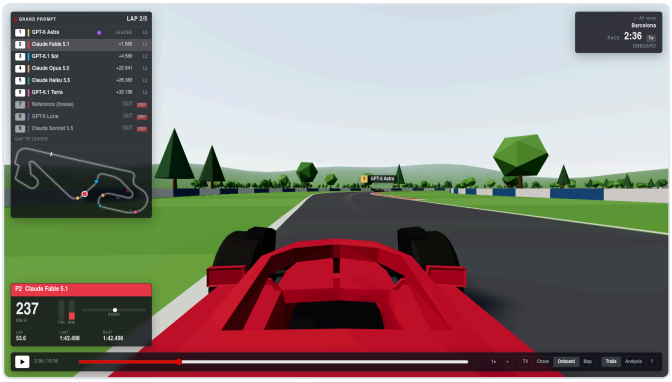
5.2 TV camera (default). A single overhead telephoto shot that never cuts, rotating slowly after the direction of travel; zoom frames the focused car plus nearby cars (owner's reference: overhead qualifying-lap comparisons). Left: timing tower (position, colour, name, gap to leader or interval, lap, fastest-lap dot, DNF badge) above a minimap with every car. Top right: track, session clock, speed, camera mode, "All races" link. Bottom left: focused-car telemetry (speed, throttle/brake bars, steer, current/last/best lap). Bottom: transport bar (play, scrub, speed 1–32×, camera, zoom slider, trails, analysis, help). Trails show each car's recent path in its colour.



5.3 TV camera on Monaco, zoomed out. Generated track mesh: asphalt ribbon, white lines, red/white kerbs on corners, grass/gravel runoff, barriers, instanced low-poly trees. The lone car in the frame is two cars apart; two of four entries are already out (DNF).



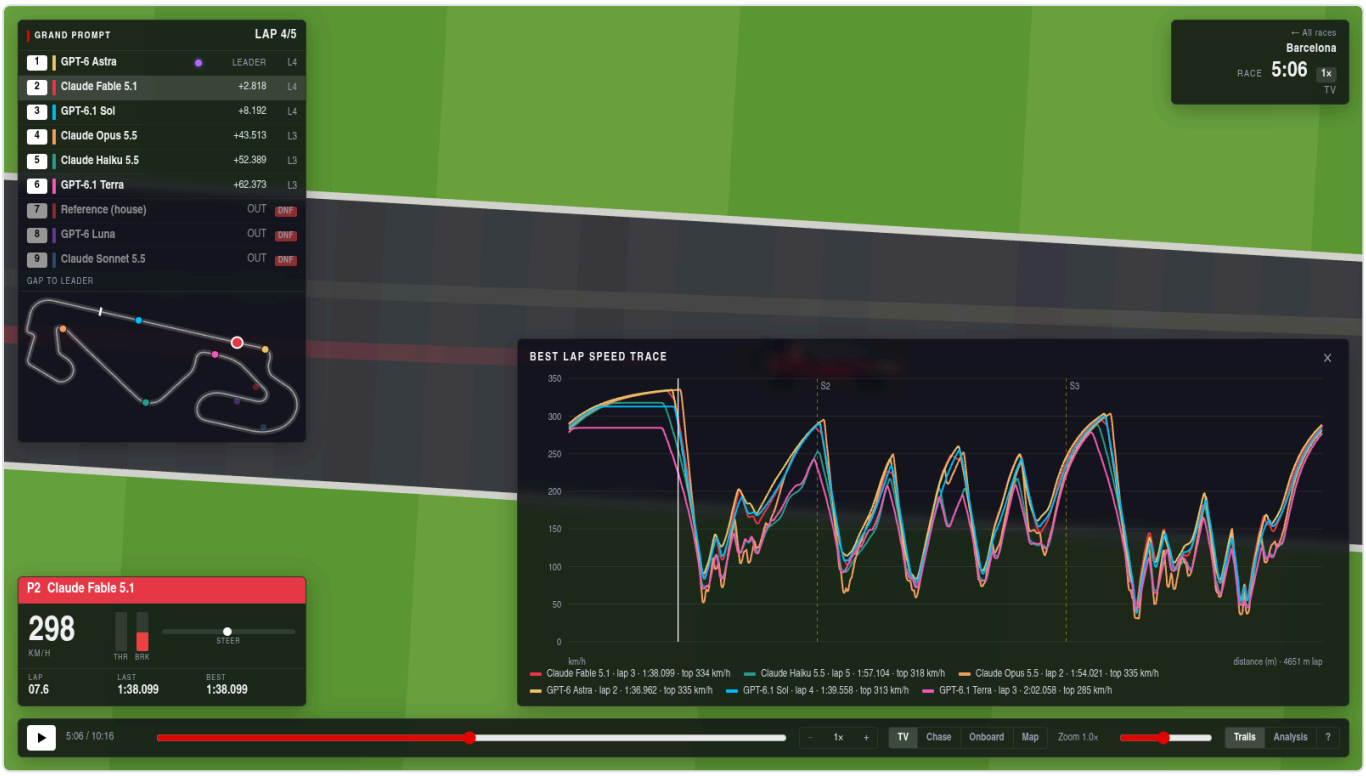
5.4 Chase camera. Behind the focused car; name labels float over other cars. Grandstand-style barriers and trees.



5.5 Onboard camera. Driver's eye view over the car body.



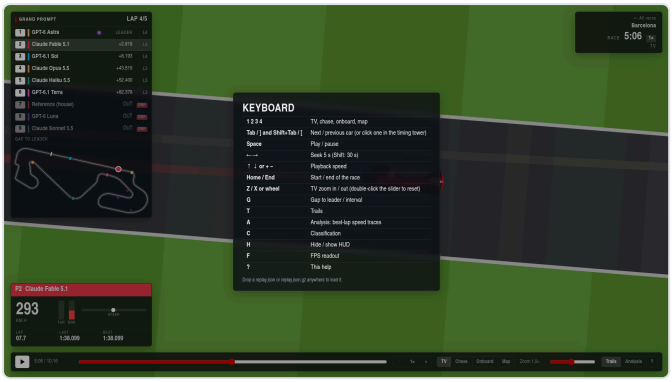
5.6 Map camera. Top-down view of the whole circuit with every car labelled by position; trails visible. Useful to read the spread of the field at a glance.



5.7 Analysis panel. Best-lap speed trace of every classified car plotted against distance, sector lines, and a legend with best lap, lap number and top speed. The cursor follows the focused car's position. It is the only comparative analysis today: no delta-time plot, no throttle/brake overlay, no sector table.



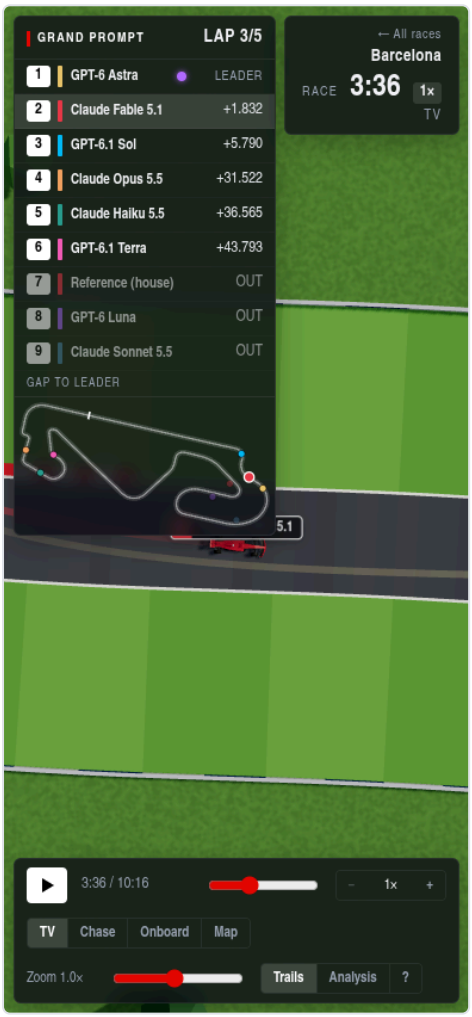
5.8 Final classification card. Opens automatically at the end, or with C. Position, colour, program name, total time, gap, best lap (purple = overall fastest), penalties, status. The message feed (top right) shows "Chequered flag". Cost and tokens, which the plan says to display, are missing here.



5.9 Keyboard help (?). The viewer is keyboard-driven; see Appendix A.



5.10 Interval mode (G). The tower toggles between gap to leader and interval to the car ahead.



5.11 Phone width (390 px). Not designed for mobile yet.

What happens on a phone today

- The timing tower and minimap cover the top half of the scene; the car is hidden behind them.
- The telemetry card disappears; the control bar wraps to three rows and takes the bottom fifth.
- No touch gestures (pinch zoom, swipe between cars); camera and car selection are keyboard-first.
- The race list is a desktop table and does not reflow.

Shared links are opened on phones first, so this is the biggest UX gap for the audience that matters most (PLAN phase 4: “mobile-friendly viewer”).

Other viewer facts

- Smooth interpolation between 50 Hz frames; target 60 fps on a laptop; FPS / draw-call readout (F).
- Deep links: `?replay=&t=&cam=&car=&speed=&zoom=&hud=&trails=&analysis=`.
- Car: an imported low-poly open-wheeler STL, decimated to 5.7k triangles and fitted to the sim’s wheelbase, painted per entry colour (in progress, uncommitted).
- Static `vite build` has no race index: only `?replay=` and drag-and-drop work when hosted statically.

6 Current visual language

Inherited, not designed: a dark broadcast style with semi-transparent blurred panels. Claude Design is free to replace it entirely, as long as the result keeps the “TV graphics” readability and stays clear of F1 branding.

TOKEN	VALUE	USE
Panel	rgba(14,16,22,.82) + blur 6px	All HUD panels, 6 px radius, hairline border rgba(255,255,255,.08)
Page background	#0b0d12	Race list, drop zone
Text / muted	#f3f4f6 / #9aa3b2	Labels, uppercase micro-headings with wide tracking
Accent	#e10600	Brand bar, scrubber, selection. Close to a well-known F1 red: worth reconsidering.
Fastest / good / bad	#b36bff · #22c55e · #ef4444	Purple = fastest lap (motorsport convention), throttle green, brake red
Font	Titillium Web → condensed fallbacks	Tabular numerals everywhere; monospace for ids/paths
Entry colours		Assigned per entry from a fixed list; no notion of a provider/team colour, and collisions happen (§7)
3D world	flat-shaded low poly	Saturated grass, dark asphalt, red/white kerbs, blue/white barriers, pale sky, soft shadows

Brand: only a wordmark “GRAND PROMPT” in small caps with a red bar. No logo, no icon, no favicon, no social preview image.

7 Known issues and gaps

#	ISSUE	DETAIL
1	Ghost cars overlap at the start	All cars start at s = 0 and sit on top of each other for about 11 s (see the label stack on the start line). A visual offset would diverge from what was timed; principle: what viewers see is exactly what was timed. Needs a design answer (grid presentation, transparency, label stacking, “ghost” styling…), owner’s call.
2	No event / leaderboard / model pages	The home is a flat list of replays, flooded by agentic practice runs. Events, standings and per-model results exist only as CLI Markdown.
3	Cost, tokens, effort, build time not shown	The plan asks for them in the classification (“they make results more interesting to share”); they are in manifests only.
4	Not mobile-friendly	See 5.11.
5	Entry colour collisions	In gp-1 the house reference and Claude Fable 5.1 share #e63946. No colour identity per provider/model family.
6	Crashes are hard to read	A DNF shows as a message and a badge; there is no highlight, no “jump to incident”, no marker on the map or timeline of where and why a car went out (power oversteer on exit, trail-braking breakaway, ECU fault).

7	Timeline has no events	The scrubber is a plain bar; laps, fastest laps, penalties and DNFs are not marked on it.
8	Dev-only bug: URL params ignored	Under React StrictMode the replay loads twice, so deep-link `t`, `cam` and `car` are dropped in the dev server.
9	Static hosting has no race index	Publishing results needs the index written as a file (or another publishing model).
10	No video output	Remotion compositions (9:16, 1:1, 16:9) planned; nothing exists. The acceptance criterion for phase 4 is a 30–60 s vertical video of a full event, rendered with no manual editing.
11	No driver insight	The code each model wrote, its strategy, its practice progression (lap time per run) and its build transcript are invisible to viewers, though they are the most interesting part of an LLM benchmark.
12	Reference driver is weak	"Gap to reference" is meaningless until the house driver is fixed; the UI should not depend on it.

8 Firm constraints

These come from the project's firm design principles. A proposal may question them, but must say so explicitly.

1. **Determinism.** A result is a pure function of (sim version, track, car, driver code, seed). Nothing in the sim or sandbox may read time or randomness.
2. **Strict separation.** The sim has no rendering dependency; the viewer contains no physics and only plays replays. **What viewers see is exactly what was timed** (no cosmetic repositioning of cars).
3. **Fairness.** Every driver gets the same information, budget, sandbox and rules; the UI must not imply otherwise (e.g. unequal budgets must be labelled).
4. **Versioned results.** Every result shows (or can show) sim, API and DDK version; past results must stay interpretable.
5. **No F1 branding.** No F1 logos, team names, liveries or look-alike trademarks. Generic open-wheel cars.
6. **Licensing.** Track data is ODbL (OSM) and LGPL (TUMFTM): attribution must be displayable. 3D assets must allow redistribution.
7. **Tech stack.** TypeScript strict, pnpm monorepo, Vite + React + react-three-fiber/drei for the viewer, Remotion (+ `@remotion/three`) for video. The sim package is browser-safe and can be reused by any UI.
8. **Quality.** ≥ 90 % coverage (lines, branches, functions, statements) on `packages/*/src`, `apps/viewer/src/lib` and `apps/viewer/server`. Designs should let logic live in pure, testable modules.
9. **Performance.** 60 fps on a laptop with ~10 cars; reasonable on a recent phone. Replays are JSON.gz, a few MB.
10. **Ghost mode until phase 5.** Cars do not interact. Phase 5 (shared world, collisions, slipstream, tyre wear, pit stops) will add overtakes and contact, and the design should anticipate it.

9 Requirements (cahier des charges)

Status: EXISTS PARTIAL MISSING. Priority: MUST SHOULD COULD. Existing features are listed too: the design should cover them, and may redesign them.

9.1 Replay viewer

ID	REQUIREMENT	STATUS	PRIORITY
V1	Load a replay from a URL parameter, from the race list, or by drag-and-drop.	EXISTS	MUST
V2	Generated 3D circuit (asphalt, lines, kerbs, runoff, barriers, start/finish, sector markers, scenery).	EXISTS	MUST
V3	Cars as generic low-poly open-wheelers, one colour per entry, name label, optional trail.	EXISTS	MUST
V4	Cameras: overhead TV (no cuts), chase, onboard, top-down map; zoom; follow any car.	EXISTS	MUST
V5	Timing tower: position, entry, gap/interval, lap, fastest lap, status; click to focus.	EXISTS	MUST
V6	Session info, focused-car telemetry, message feed of race events.	EXISTS	MUST
V7	Transport: play/pause, scrub, speed 1–32×, keyboard shortcuts, deep links to a moment.	EXISTS	MUST
V8	Timeline with event markers (laps, fastest laps, penalties, crashes, finish) and jump-to-incident.	MISSING	SHOULD
V9	Readable start in ghost mode without moving cars from their timed position (issue 1).	MISSING	MUST
V10	Final classification with time, gap, best lap, penalties, status *and* cost, tokens, wall time, practice runs.	PARTIAL	MUST
V11	Analysis: speed traces vs distance (exists); add delta-time vs a chosen car, throttle/brake overlay, sector table, mini-sectors on the map.	PARTIAL	SHOULD
V12	Incident view: where and how a car went out (map marker, short replay loop, cause in plain words).	MISSING	SHOULD
V13	Mobile layout: portrait-first HUD, touch controls (tap tower to focus, swipe cars, pinch zoom), collapsible panels.	MISSING	MUST
V14	Attribution / about panel (track data licences, versions, seed, state hash, “reproduce” instructions).	PARTIAL	SHOULD
V15	Qualifying presentation distinct from race (one flying lap each: ideal for overlay / “lap battle” comparisons).	MISSING	COULD

9.2 Browsing, events and results

ID	REQUIREMENT	STATUS	PRIORITY
B1	List of every replay, filterable, newest first.	EXISTS	MUST
B2	Event page: format (practice tracks, event track, budget, time limit, effort), qualifying + race results, cost and tokens per model, links to both replays.	MISSING	MUST
B3	Leaderboard / season standings across events (points, wins, DNFs, total cost).	MISSING	MUST
B4	Model (entry) page: manifest, practice history (lap time per run per circuit), final driver code with stats, build transcript/log, results across events.	MISSING	SHOULD
B5	Practice runs grouped under their build instead of flooding the home list.	MISSING	MUST
B6	Circuit pages: map, length, corners, which events used it, lap records.	MISSING	COULD
B7	Works when hosted statically (index generated at publish time) as well as in the dev server.	PARTIAL	SHOULD

B8	Clear caveats on results that are trials (unequal budgets, effort not fixed, e.g. gp-1).	MISSING	SHOULD

9.3 Shareable content

ID	REQUIREMENT	STATUS	PRIORITY
S1	Video export in 9:16, 1:1, 16:9, rendered deterministically from a replay: intro card (“8 AI models, one circuit”), race with camera direction, highlight callouts (crash, fastest lap, track limits; overtakes in phase 5), final card with times and costs.	MISSING	MUST
S2	Automatic highlight detection from events (biggest crash, fastest lap, closest gap, comeback) feeding both the video and the viewer timeline.	MISSING	SHOULD
S3	Social preview image per replay/event (Open Graph) and short deep links to a moment.	MISSING	SHOULD
S4	Still “result card” export (PNG) for posts: podium, times, costs.	MISSING	COULD

9.4 Operator tooling (owner-facing)

ID	REQUIREMENT	STATUS	PRIORITY
O1	CLI with human- and LLM-readable output (validate, practice, qualify, race, event, leaderboard, serve, workspace).	EXISTS	MUST
O2	Build monitor: progress of each model during a parallel build (status, elapsed vs 30 min, practice runs used, last lap time).	MISSING	COULD
O3	Practice telemetry plots (speed / throttle / brake / steer / lateral offset per lap) for debugging drivers.	MISSING	COULD

10 Open areas: proposals welcome

Claude Design is explicitly invited to propose refactors and new features. Some directions the owner is curious about, none of them decided:

- **Identity.** Name, logo, wordmark, colour system that reads as motorsport without borrowing F1’s look. A colour identity per model family/provider that stays distinct with 8–12 entries and in colour-blind conditions.
- **Information architecture.** Home → events → race; seasons; models; circuits. Where the replay viewer sits in it. Whether the viewer and a results site should be one app or two.
- **The “engineer” story.** Making the LLM’s work visible: its code (size, structure, the approach it chose: precomputed racing line vs solved in `init`), its practice progression, its cost. This is what distinguishes the benchmark from a racing game.
- **Ghost-mode presentation.** How to show overlaid independent runs honestly: ghost transparency, distance-based “virtual gaps”, delta bars, split-screen lap comparison.
- **Mystery GP reveal.** The event circuit is unseen by every model; how to stage the reveal and the generalisation story (practice circuits vs event circuit).
- **Broadcast graphics.** Lower thirds, battle graphics, sector colours (purple/green/yellow), DNF cards with cause, lap counter, final podium.
- **Video templates.** Composition and pacing for 30–60 s vertical clips generated with no manual editing.
- **Phase 5 readiness.** How the UI grows when cars race wheel-to-wheel: overtakes, contact, tyre wear/temperature, pit stops, strategy.

- **Code refactors.** e.g. a shared UI component library between viewer and video; a results “publishing” step producing a static index and per-event JSON; separating the HUD from the 3D scene state (today a mutable `live` ref polled at 10 Hz).

---

11 Expected deliverables from Claude Design

1. **Design system:** tokens (colour incl. entry palette, type scale with tabular numerals, spacing, radii, elevation/blur, motion), light/dark if relevant, and components (timing tower row, status badges, gap chips, telemetry gauges, message toast, transport bar, cards, tables, charts).
2. **Screens** at desktop (1600×900) and phone (390×844): home, event, leaderboard, model page, replay viewer (each camera mode, start of race, mid-race, incident, finish), classification, analysis.
3. **Video frames:** key frames for 9:16 (intro, race, highlight, final card), and 16:9 if different.
4. **Flows:** link shared on a phone → watch → understand result; owner runs an event → publishes it.
5. **Proposals list:** each refactor or feature with motivation, impact on firm constraints (if any), rough size, and a suggested order. These will be reviewed before implementation.

> The output is implemented by a coding agent in this repository (TypeScript, React, react-three-fiber, Remotion). Designs that map to concrete components and tokens, with states (empty, loading, error, DNF, many cars, long names) spelled out, are the most useful.

A Appendix

A.1 Keyboard shortcuts

KEY	ACTION
1 2 3 4	TV, chase, onboard, map camera
Tab /] · Shift+Tab / [	Next / previous car (or click in the timing tower)
Space	Play / pause
← → (Shift)	Seek 5 s (30 s)
↑ ↓ or + −	Playback speed (1, 2, 4, 8, 16, 32×)
Home / End	Start / end of the session
Z / X, mouse wheel	TV zoom in / out (0.4–2.8×)
G · T · A · C · H · F · ?	Gap/interval · trails · analysis · classification · HUD · FPS · help

A.2 URL parameters

`replay` (URL of replay.json[.gz]) · `t` (s) · `cam` (tv|chase|onboard|map) · `car` (entry id) · `speed` · `play` · `zoom` · `hud` · `trails` · `analysis` · `fps`.

A.3 Race statuses and event messages

Statuses: RUNNING, FINISHED, DNF: CRASHED (barrier above 12 m/s), DNF: STALLED (< 1 m/s for 10 s), DNF: ECU FAILURE (driver code errors: 3 strikes, or any failure in init), TIME UP.

Messages: lap deleted (track limits), fastest lap, track-limits warning (first 3 in a race), +5 s penalty (after that), crash at X km/h, barrier contact at X km/h, driver code fault, chequered flag, DNF.

Rules: off track = car centre more than 1.0 m beyond the white line; any off-track moment invalidates the lap; a lap counts only if every sector checkpoint was crossed in order.

A.4 Driver API (what an LLM implements)

```
export function init(track: TrackInfo, car: CarSpec): void;   // once; precompute (budget: 20k units)
export function step(t: Telemetry): Controls;                 // 50 Hz (budget: 20 units)

Telemetry: t, lap, lapTime, sector, s, n, headingError, x, y, yaw, speed, vx, vy, yawRate,
           ax, ay, steer, throttle, brake, surface, offTrackWarnings
Controls:  throttle 0..1, brake 0..1, steer -1..1 (fraction of maxSteer, + = left)
CarSpec:   mass, wheelbase, width, length, weightFront, maxSteer, steerRate, throttleRate,
           brakeRate, maxPower   (tyre and aero constants hidden: information level B)
```

A.5 Replay format (excerpt)

```
{ "format": "bench-replay", "formatVersion": 1,
  "header": { simVersion, apiVersion, ddkVersion, trackId, trackHash, carId, carHash, seed,
              session: { type, laps, seed, infoLevel }, frameRate: 50, stateHash,
              car: { length, width, wheelbase }, referenceLap? },
  "track":  { id, name, length, spacing, sectors, x[], y[], heading[], curvature[],
              widthLeft[], widthRight[], kerbLeft[], kerbRight[], runoffType*[], runoffWidth*[] },
  "cars":   [ { id, name, color, driverHash,
                frames: { x[], y[], yaw[], speed[], steer[], throttle[], brake[], surface[], lap[], s[]
},
                events: [ { t, type, lap, value?, detail? } ] } ],
  "classification": [ { position, id, name, color, status, lapsCompleted, laps[], bestLap,
bestLapNumber,
                      finishTime, penalties, totalTime, gap, offTracks, warnings, strikes, maxSpeed } ]
}
```

A.6 Manifest (per model and event)

```
{ "model": "Claude Fable 5.1",
  "settings": { modelId, cli: "claude"|"codex"|"gemini", practiceTracks: [...], eventTrack,
                sandbox: "bubblewrap", timeLimitMin: 30, outcome, driverSource, turns, effort },
  "usage":    { inputTokens, outputTokens, costUsd?, wallTimeS, practiceRuns } }
```

A.7 Vehicle (for context)

800 kg open-wheeler, dynamic bicycle model, simplified Pacejka tyres with friction ellipse, aero drag and downforce, actuator rate limits. Measured: top speed 341 km/h, 0–100 km/h in 2.56 s, peak braking 5.26 g, 1.9 g lateral at 72 km/h and 4.2 g at 306 km/h. Full throttle below ~190 km/h spins the rear: traction control is part of the skill.

A.8 Glossary

Driver / program	The JS module an LLM wrote; it is what races, not the LLM.
Entry	One model's submission in an event.
Event	Build + qualifying + race for a set of models on one circuit (e.g. gp-1).
Ghost mode	Each car runs alone; the replay overlays them.
Agentic build	The model works in a coding-agent CLI with practice runs, versus one-shot (single prompt).
House / reference driver	Internal driver used to detect exploits; not a ceiling.
Information level B	Tyre and aero constants hidden; models infer them from practice telemetry.
Frenet (s, n)	Distance along the centreline, signed lateral offset (+ = left).
Mystery GP	Racing on a circuit no model practised on, to measure generalisation.